

Streaming versus non-streaming calls

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 7 of 12 · English notes

Until mid-session, every demo used `messages.create` and waited for a finished **Message**. Lalit asked how Claude Chat shows tokens as they appear. Vijay's answer: use the SDK `stream` helper or set `stream=true` on `create`. Same model, different delivery: one JSON blob versus a sequence of **server-sent events**.

Non-streaming: one Message when generation ends

With default `create`, the HTTP response body is the full assistant message. If generation took “a minute” (Vijay's exaggerated example), the client UI would stay blank for that minute. You then parse `content` as complete text blocks (`type: text`). This is simpler for batch jobs, tests, and any caller that cannot render partial text. It is a **synchronous** wait from the application's point of view: the call returns only when the model is done (or hits `max_tokens / stop`).

Why stream at all

- Perceived latency: show tokens as soon as they exist so users stay engaged.
- Long answers: do not buffer the entire essay in the SDK before painting the first sentence.
- Chat-like products: Claude Chat and IDE agents feel fast because they stream; your API app will feel “stuck” if you only use blocking `create`.

Streaming does not make the model smarter or cheaper by itself. You still pay for output tokens. You *do* take on event-handling complexity.

Two SDK entry points

Mechanism (class language)	Idea	Typical use
<code>messages.create</code> without <code>stream</code>	Full Message object	Scripts, evals, simple backends
<code>stream=true</code> on <code>create</code>	Same method, streamed HTTP	When you want <code>create</code> but incremental bytes
<code>messages.stream</code> / <code>stream</code> helper	Dedicated streaming API in the SDK	Event callbacks, context managers

Vijay said each path has benefits; pick based on whether your runtime wants a finished object or an iterator of events. Confirm the exact method names on the current Python/TS SDK — they have been renamed more than once.

Events you were shown

On the wire, streaming is a series of events. The notebook printed them all, then filtered to text. Remember the names in class terms and map them to docs:

1. **message_start** — a message id exists; content is still empty. “I am about to send data.”
2. **content_block_start** — a new block (usually text) begins.
3. **content_block_delta** — incremental payload. Deltas accumulate. Text arrives as **text_delta** pieces, not a finished type: text block yet.
4. **content_block_stop** — that block is complete.
5. **message_stop** (class: raw message stop) — the stream for this message is finished.

End users should not see event names. Your UI concatenates `text_delta` values. If you ignore events other than text, you will miss errors, usage, and (later) tool or thinking blocks.

“THINKING...” IN VS CODE IS NOT THIS STREAM

Mritunjay asked which block holds the humanised “thinking / working” chrome in VS Code. Vijay: that UX is **Agent SDK** / product layer, not the raw Messages stream. Agent SDK still sits on these deltas for the actual tokens.

Stop reasons still apply

Whether streamed or not, generation can end because of end-of-turn, a **stop sequence** (next chapter), or **max_tokens**. Streaming simply delivers the same tokens earlier. If `max_tokens` fires, the last deltas may be a truncated sentence — handle incomplete JSON yourself.

Batches (awareness only)

The messages resource also exposes **batches** for offline / bulk jobs. Foundation focus in this hour was interactive create versus stream, not designing a batch pipeline. Know that batch exists so you do not confuse it with streaming.

DOCS OVER NOTEBOOK EVENT DUMPS

Event type strings in August 2026 cells may already differ. Use [streaming docs](#) as the scoring key.

Implementation checklist

- Blocking create: `parse_content[ ].text`.
- Streaming: subscribe to deltas; join text; still read final usage/stop from the terminal event or a follow-up Message.
- Do not assume streaming reduces billed tokens.
- Time-to-first-token is the UX metric streaming improves.

Official documentation

- **Streaming**

<https://docs.claude.com/en/docs/build-with-claude/streaming>

Event types, SDKs, and SSE behaviour.

- **Messages API**

<https://docs.claude.com/en/api/messages>

The `stream` field on create.

- **API overview**

<https://docs.claude.com/en/api/overview>

How clients wrap HTTP including streamed responses.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.